

String in C# with Examples

In this article, I am going to discuss **String in C#** with Examples. Please read our previous article, where we discussed [Command Line Arguments in C#](#) with Examples. As a developer, it is very important to understand the concept of Strings in C# and I am also sure that you are using the string in all of your projects. But there are many things that you should know from a performance point of view. So, as part of this article, we are going to discuss the following pointers in detail with examples.

1. **Strings are Reference Types**
2. **Understanding the difference between string(small) vs String(Capital).**
3. **Strings are immutable.**
4. **How we can improve performance using String intern?**
5. **StringBuilder for concatenation.**
6. **Why do they make strings immutable?**

What is a String?

In C#, the string is an object of the String class that represents a sequence of characters. We can perform many operations on strings such as concatenation, comparison, getting substring, search, trim, replacement, etc.

Strings are reference types in C#:

Strings in C# are reference types i.e. they are not normal data types or you can say they are not like other primitive data types. For example, if we define some variables using int or double data types as shown below.

```
int i = 10;  
double d = 20.5;
```

Then if you right-click on the data type and go to the definition then you will see that they are struct as shown in the below image. Struct means they are value type.

```
public struct Double : IComparable, IFormattable, IConvertible, IComparable<Double>, IEquatable<Double>  
{
```

On the other hand, if you define a variable with string data type as shown below.

```
string str = "";
```

Then if you right-click on the string data type and click on go to definition then you will see that it is a class. Class means reference data type.

```
public sealed class String : IComparable, ICloneable, IConvertible, IEnumerable, IComparable<String>,
    IEnumerable<char>, IEquatable<String>
```

So, the first point that you need to remember is strings are reference types while other primitive data types are struct types i.e. value type in C#.

What are the Differences between String(Capital) vs string(small) in C#?

In C#, you can use the string in two ways i.e. you can use the string using capital S (i.e. String) or by using the small "s" (i.e. string) as shown in the below image.

```
string str1 = ""; //using small s
String str2 = ""; //using capital S
```

Now the question that should come to your mind is what is the difference between these two (string vs String) in C#. Let's understand this. The small string is actually an alias of String (Capital string). If you right-click on the small string and if you go to the definition then you will see that the actual class name is a capital string i.e. String as shown in the below image.

```
public sealed class String : IComparable, ICloneable, IConvertible, IEnumerable, IComparable<String>,
    IEnumerable<char>, IEquatable<String>
```

You can use any one of them i.e. either string or String. But as per the naming convention when you are creating a variable use the small string (i.e. string) and whenever you want to invoke methods on the string then use the capital string (i.e. String) as shown in the below image.

Use small string to declare variable

```
↓
string str2 = String.Concat(" ");
```

↑
Use Capital String to invoke method

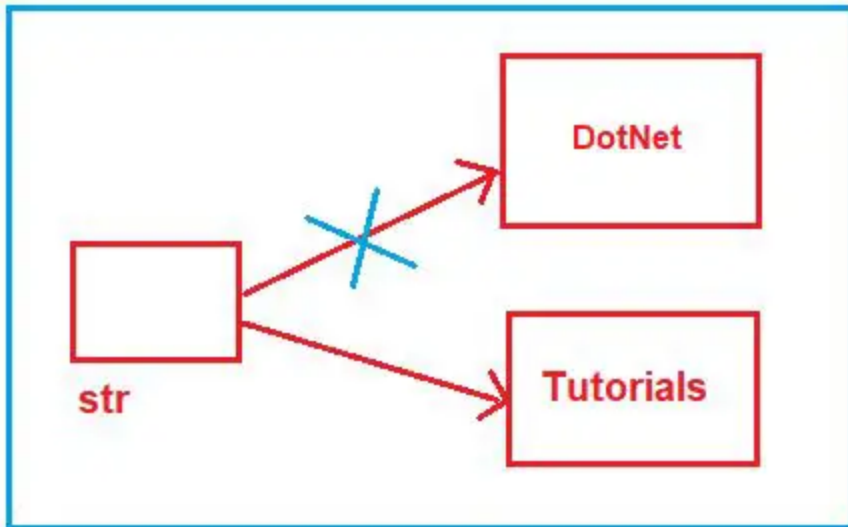
Strings are Immutable in C#:

Before understanding strings are immutable, first, we need to understand two terms i.e. Mutable and Immutable. Mutable means can be changed whereas Immutable means can not be changed. C# strings are immutable means C# strings cannot be changed. Let us understand this with an example.

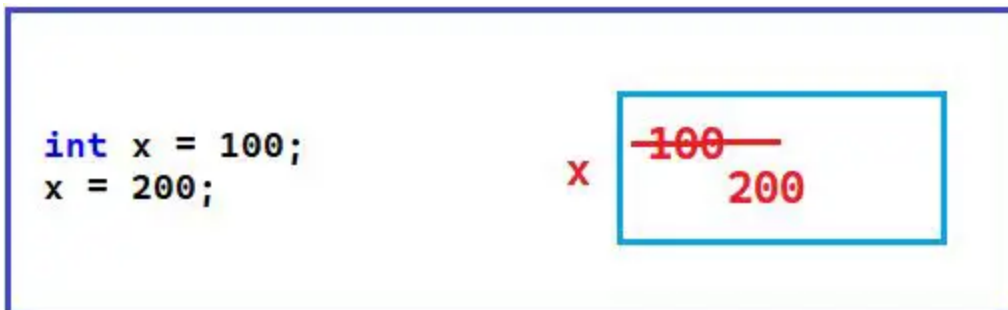
Please have a look at the below image. When the first statement is executed, it will create one object and assign the value DotNet. But when the second statement is executed, it will not override the first object, it lets the first object be there for garbage collection and creates a fresh object, and assign the value Tutorials.

```
string str = "Dotnet"; //Creates one object of string and assign the value "Dotnet"
str = "Tutorials"; //creates a fresh object and assign the value "Tutorials"
```

So, when the above two statements are executed, internally two memory locations are created. When the first statement is executed, one object will be created which holds the value DotNet and that object will be referred to by the str variable. When the second statement will be executed, another object will be created which holds the value Tutorials and now the str variable will point to this newly created object. And the first object will be there and will be available for garbage collection. So, the point that you need to remember is that each time, we assign a new value to the string variable, a new object is created and that new object will be referred to by the string variable and older objects will be there for garbage collection and this is the reason why said strings are immutable in C#.



But this is not the case with a value type. For example, please have a look at the below two statements. When the first statement is executed one memory location is created and assigned the value 100 and when the second statement is executed, it will not create a new memory location rather it will override the value of the same memory location.



Note: The point that you need to remember is with the value data type, you can override the value of the same memory location and hence they are said to be **Mutable**. But with the string data type, you cannot modify the value of a memory location, and hence strings are said to be **Immutable**.

Example to Proves C# strings are Immutable:

Let us see an example to understand C# strings are Immutable. Please copy and paste the following code. As you can see here we have a heavy loop. As part of the Loop, we assign a value to the string str variable. Here, we are using GUID to generate a new value, and each time it will create a new value and assign it to the str variable. Again, we are using Stopwatch to check how much time it took to execute the loop.

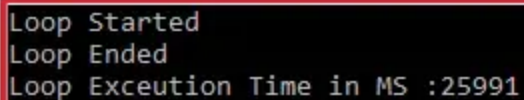
```
using System;
```

```

using System.Diagnostics;
namespace StringDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            string str = "";
            Console.WriteLine("Loop Started");
            var stopwatch = new Stopwatch();
            stopwatch.Start();
            for (int i = 0; i < 30000000; i++)
            {
                str = Guid.NewGuid().ToString();
            }
            stopwatch.Stop();
            Console.WriteLine("Loop Ended");
            Console.WriteLine("Loop Execution Time in MS :" +
stopwatch.ElapsedMilliseconds);
            Console.ReadKey();
        }
    }
}

```

Output: When you execute the program, you will get the following output. The time may vary in your machine.



```

Loop Started
Loop Ended
Loop Execution Time in MS :25991

```

As you can see in the above output, it approximately took 26000 milliseconds to execute the loop. Each time the loop executes, it creates a fresh string object and assigns a new value to it. This is because strings are immutable in C#.

Example using Integer in C#:

In the following C# example, instead of a string, we are using an integer variable. As integers are not immutable, so it will not create a fresh memory location each time the loop executes instead it will use the same memory location and update its value.

```

using System;
using System.Diagnostics;
namespace StringDemo
{
    class Program
    {
        static void Main(string[] args)
        {

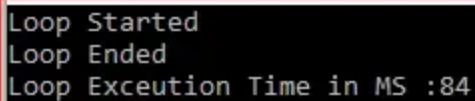
```

```

        int ctr = 0;
        Console.WriteLine("Loop Started");
        var stopwatch = new Stopwatch();
        stopwatch.Start();
        for (int i = 0; i < 30000000; i++)
        {
            ctr = ctr + 1;
        }
        stopwatch.Stop();
        Console.WriteLine("Loop Ended");
        Console.WriteLine("Loop Execution Time in MS :" +
stopwatch.ElapsedMilliseconds);
        Console.ReadKey();
    }
}

```

Output:



```

Loop Started
Loop Ended
Loop Execution Time in MS :84

```

As you can see in the above output, it only took 84 milliseconds to execute the loop.

Example: String with Same value in C#

Let us understand what will happen if we assign the same value to the string variable again and again with an example in C#. As you can see in the below example, which is exactly the same as the first example, but here instead of using GUID, we are assigning a fixed value to the string str variable.

```

using System;
using System.Diagnostics;
namespace StringDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            string str = "";
            Console.WriteLine("Loop Started");
            var stopwatch = new Stopwatch();
            stopwatch.Start();
            for (int i = 0; i < 30000000; i++)
            {
                str = "DotNet Tutorials";
            }
            stopwatch.Stop();
            Console.WriteLine("Loop Ended");
        }
    }
}

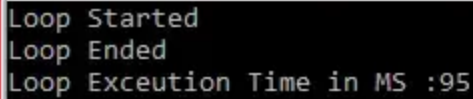
```

```

        Console.WriteLine("Loop Exceution Time in MS :" +
stopwatch.ElapsedMilliseconds);
        Console.ReadKey();
    }
}
}

```

Output:



```

Loop Started
Loop Ended
Loop Exceution Time in MS :95

```

As you can see in the above output it only took 95 milliseconds. This is because in this case fresh objects are not created each time the loop executes. Now, the question that should come to your mind is why? The answer is **String intern**. So, let us understand string interning in detail.

String Intern in C#:

The **String Intern in C#** is a process that uses the same memory location if the value is the same. In our example, when the loop executes for the first time, it will create a fresh object and assign the value “**DotNet Tutorials**” to it. When the loop executes 2nd time, before creating a fresh object, it will check whether this “**DotNet Tutorials**” value is already there in the memory, if yes then it simply uses that memory location else it will create a new memory location. This is nothing but C# string interning.

So, if you are running a for loop and assigning the same value again and again, then it uses string interning to improve the performance. In this case, rather than creating a new object, it uses the same memory location. But when the value changes it will create a new fresh object and assign the value to the new object.

StringBuilder for Concatenation in C#:

As we already discussed if the value changes then every time it will create a new fresh object in C# and this is because of the Immutability behavior of the string. The C# string immutability behavior can be very very dangerous when it comes to string concatenation. Let us understand string concatenation in C# with an example and understand the problem. In the below example, we are concatenating the string using the for loop.

```

using System;
using System.Diagnostics;
namespace StringDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            string str = "";
            Console.WriteLine("Loop Started");
            var stopwatch = new Stopwatch();
            stopwatch.Start();

```

```

        for (int i = 0; i < 30000; i++)
        {
            str = "DotNet Tutorials" + str;
        }
        stopwatch.Stop();
        Console.WriteLine("Loop Ended");
        Console.WriteLine("Loop Exceution Time in MS :" +
stopwatch.ElapsedMilliseconds);
        Console.ReadKey();
    }
}
}

```

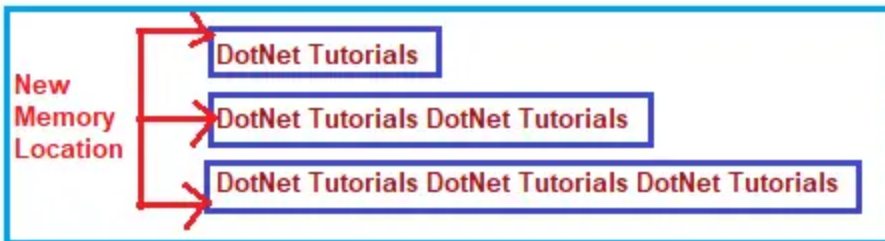
Output:

```

Loop Started
Loop Ended
Loop Exceution Time in MS :5473

```

As you can see in the above image, it took approximately 5473 milliseconds to execute the loop. In order to understand how it executes the loop, please have a look at the below image. The loop executes the first time, it will create a new memory location and store the value "DotNet Tutorials". For the second time, it creates another fresh memory location (fresh object) and stores the value "DotNet Tutorials DotNet Tutorials" and the first memory location will be going for garbage collection. And the same process will continue i.e. each time the loop executes a new memory location will be created and previous ones will be going for garbage collection.



In order to solve the above **String Concatenation Problem in C#**, the .NET Framework provides the **StringBuilder** class. As the name itself says everything, the string builder class in C# is used to build a string by concatenating the strings. If you use string builder then fresh objects are not going to be created every time you concatenate something to the string variable in C#.

Example using StringBuilder in C#:

Let us understand how to overcome the **String Concatenation Problem in C#** using the **StringBuilder** class. In the following example, we are using the **StringBuilder** class to concatenate strings. Here, first, we create an instance of the **StringBuilder** class and then use the **Append** method of the **StringBuilder** class to concatenate the string.

```

using System;
using System.Diagnostics;
using System.Text;

```

```

namespace StringDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            StringBuilder stringBuilder = new StringBuilder();
            Console.WriteLine("Loop Started");
            var stopwatch = new Stopwatch();
            stopwatch.Start();
            for (int i = 0; i < 30000; i++)
            {
                stringBuilder.Append("DotNet Tutorials");
            }
            stopwatch.Stop();
            Console.WriteLine("Loop Ended");
            Console.WriteLine("Loop Execution Time in MS :" +
stopwatch.ElapsedMilliseconds);
            Console.ReadKey();
        }
    }
}

```

Output:

```

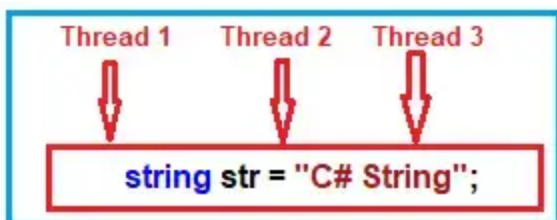
Loop Started
Loop Ended
Loop Execution Time in MS :1

```

As you can see in the above output, it only took 1 millisecond to concatenate the string compared to 5473 milliseconds using string. This is because every time the for loop runs it will not create fresh objects rather than it will use the same memory location i.e. the same old object which drastically improves the application performance.

Why do they make C# String Immutable?

Now the question is why they made strings as Immutable in C#. **They made Strings Immutable for Thread Safety.** Think of one situation where you have many threads and all the threads want to manipulate the same string object as shown in the below image. If strings are mutable then we have thread-safety issues.



In case you are new to thread safety, I strongly recommended you read the following article, where we discussed Thread and Thread Safety in detail.

<https://dotnettutorials.net/lesson/multithreading-in-csharp/>

In the next article, I am going to discuss **Static in C#** with examples. Here, in this article, I try to explain **Strings in C#** with Examples and I hope you enjoy this Strings in C# with examples article. I would like to have your feedback. Please post your feedback, question, or comments about Strings in C# with Examples article.